Distributed Chat Application - Documentation

By UMER Muhammad and GOMEZ ROSALES Laura Sofía

Content Table

Distributed Chat Application - Documentation **1**
1. Introduction **1**
2. Global Functionality of the Application **2**
3. Software Architecture **2**
3.1 Architectural Style 2
3.2 Component Responsibilities 3
Server 3
Client 3
3.3 Communication Model 3
4. Key Architectural Decisions **4**
5. How to clone the Repository **4**
6. How to run the code **4**
6.1 Prerequisites **4**
6.2 Compilation 4
6.3 Start the Server **5**
6.4 Start Clients **5**
7. Commands **5**
8. Conclusion **5**

1. Introduction

This project implements a distributed chat application using Java RMI (Remote Method Invocation). With the main objective of design and implementing a distributed client-server application with proper architectural structure, synchronization, and remote communication.

The application enables multiple users to communicate in real time through a centralized server. The system supports user join and leave notifications, message broadcasting, persistent chat history, and controlled server shutdown.

2. Global Functionality of the Application

The chat application provides the following global functionalities:

- Users can connect to a central server using a username
- When a user joins, a system notification is generated and broadcast
- Clients with duplicate usernames are not allowed
- Users can send messages that are broadcast to all connected clients
- Messages include timestamps
- The server maintains a persistent history of chat messages
- When a user leaves, a system message is recorded and broadcast
- The server handles unexpected disconnections
- On server shutdown, connected clients are notified and exit gracefully
- Empty messages are ignored and not broadcast.

From a user perspective, the system behaves like a real-time shared communication room where all events (join, message, leave) are recorded.

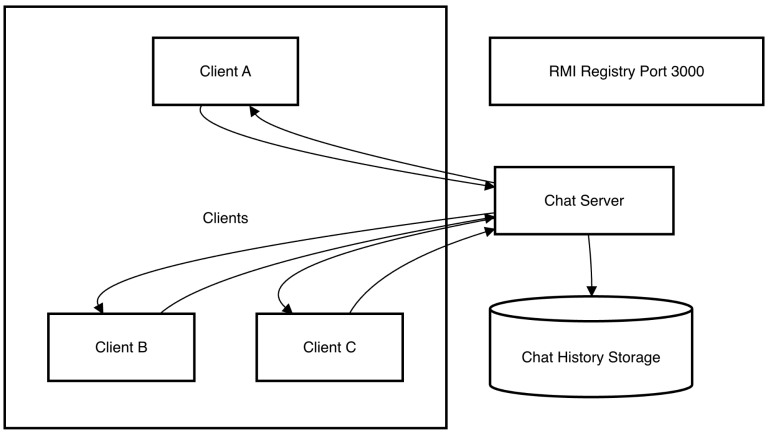
3. Software Architecture

The application is built as a Coordination-based Distributed System using Java RMI. It is designed to hide the complexities of distribution while ensuring a consistent global state.

3.1 Architectural Style

The system follows a Client-Server architecture where the Server acts as a Central Coordinator.

- There is no direct communication between clients. By using the Server as an intermediary, we achieve high spatial decoupling; Client A does not need to know the identity or location of Client B to interact with them
- Through the RMI Registry, the system hides the physical location of the server object. The client interacts with a symbolic name "ChatService"



3.2 Component Responsibilities

Server

The server is responsible for:

- Managing connected users
- Receiving messages from clients
- Broadcasting messages to all clients
- Generating timestamps
- Maintaining persistent chat history
- Handling join and leave events

Client

Each client is responsible for:

- Connecting to the server
- Sending user messages
- Receiving broadcast messages
- Displaying messages in chronological order

3.3 Communication Model

The application uses remote method invocation (RMI) for communication.

Communication occurs in two directions:

1. Client → Server (Request)
    - Join chat
    - Send message
    - Leave chat
2. Server → Client (callback)
    - Deliver broadcast messages

Before allowing a user to enter, the server checks its internal Map for the chosen username.

- If the username exists, the server performs a Liveness Check (Ping) by attempting a remote call to the existing client stub
- If the old client is unresponsive, the server automatically cleans up the stale reference. This ensures the system state is always synchronized with the physical reality of the network. If the user is truly active, the join is rejected to maintain State Integrity

Them the new client invokes getHistory() to to catch up with all past coordination events stored in the server's memory and disk.

Finally, the server adds the client to the active group and broadcasts the "Join" event, so when a user sends a message:

1. The client sends it to the server
2. The server timestamps and stores it
3. The server broadcasts it to all connected clients

This ensures consistent ordering of messages.

4. Key Architectural Decisions

By using a centralized server as a Sequencer, we achieve Total Ordering of messages. Even if two clients send messages at the exact same physical time, the Server’s synchronized methods force a FIFO (First-In-First-Out) queue, ensuring all clients see the same sequence of events

Also, Instead of clients "polling" the server (which is inefficient), we use a Callback Pattern. The server holds a remote reference to each client and "pushes" notifications. This is an implementation of the Publish-Subscribe model

5. How to clone the Repository

```
git clone https://github.com/Slrosales/p_chat.git
cd p_chat
```

6. How to run the code

6.1 Prerequisites

- Linux environment (Ubuntu recommended)
- ava Development Kit (JDK 17+)
- Port 3000 must be available
- If you switch Java versions, delete history.bin to avoid InvalidClassException

6.2 Compilation

Use the provided build script to clean and compile all sources:

```
chmod +x build.sh
./build.sh
```

6.3 Start the Server

Starts the RMI Registry and the Server application. The server creates history.bin in the working directory.

```
./run_server.sh
```

6.4 Start Clients

In terminal mode:

```
./run_client.sh Laura
./run_client.sh Umar
```

To run on GUI mode:

```
java -cp classes src.ChatGUI Laura
java -cp classes src.ChatGUI Umar
```

or, if you want to specify the port:

```
./run_gui.sh Laura 3000
./run_gui.sh Umar 3000
```

This will connect to the server running on `localhost` at port `3000`. Make sure the server is running in the same port before starting the clients.

7. Commands

- **Client:** Type any text to send. Type `/leave` to exit gracefully.
- **Server:** Type `/exit` or press `Ctrl+C` to stop (both trigger the persistence save).

8. Conclusion

The application implements a distributed chat system using a centralized client-server architecture. The server coordinates communication, maintains consistency, and ensures reliable message broadcasting. The design separates client and server responsibilities clearly while supporting multiple concurrent users.